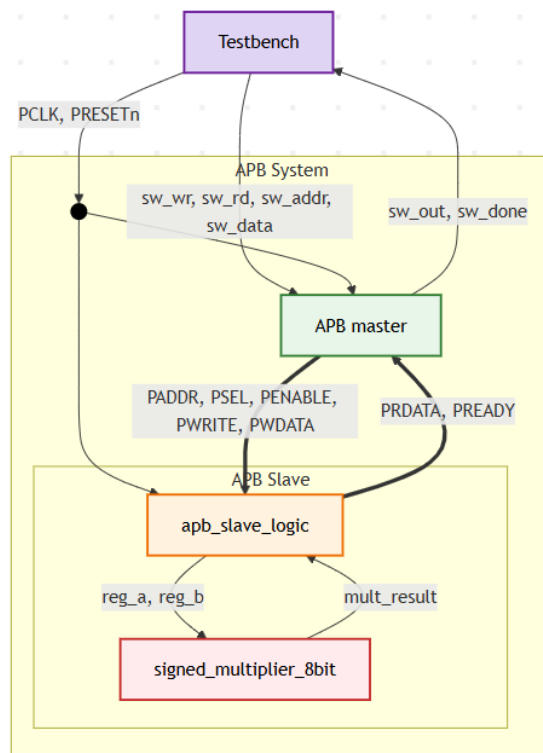


Thiết kế hệ thống SoC đơn giản và hệ thống Custom Bus trong SoC

- Module và bus
 - Thiết kế lựa chọn:
 - Module nhân 2 số 8 bit có dấu
 - Bus APB
 - Lý do chọn bus APB cho module nhân Phù hợp cho thiết kế đơn giản: không có pipeline, thiết lập địa chỉ 1 chu kỳ, truyền dữ liệu 1 chu kỳ → giúp tiết kiệm tối đa diện tích và năng lượng so với các bus phức tạp.
- Tổng quan hệ thống



Hình 1 Hệ thống SoC cụ thể cho thiết kế

- Testbench sẽ gửi tín hiệu CLK và Reset cho cả APB master và APB slave.
 - APB master nhận tín hiệu write, read và địa chỉ, dữ liệu từ Testbench, sau đó sẽ truyền tín hiệu PADDR, PSEL, PENABLE, PWRITE, PWDATA để slave hoạt động
 - Slave sẽ truyền giá trị a và b vào module nhân và nhận kết quả từ module đó. Sau đó truyền kết quả và cờ báo chuẩn bị PREADY về để APB Master tiếp tục truyền tín hiệu.
 - Master sẽ đưa tín hiệu cờ báo done và kết quả về Testbench.
- Thiết kế các module cụ thể

```

module signed_multiplier_8bit (
    input wire signed [7:0] a,
    input wire signed [7:0] b,
    output wire signed [15:0] p
);
    assign p = a * b;
endmodule

```

Bảng 1 Module nhân

```

module apb_slave (
    input wire    PCLK,
    input wire    PRESETn,

    // Các tín hiệu chuẩn của bus APB
    input wire    PSEL, // Chip Select: Kích hoạt module này
    input wire    PENABLE, // Enable: Tín hiệu xác nhận của chu kỳ bus
    input wire    PWRITE, // Write Enable: 1 = Ghi, 0 = Đọc
    input wire [7:0] PADDR, // Address: Bus địa chỉ
    input wire [31:0] PWDATA, // Write Data: Bus dữ liệu ghi vào

    output wire    PREADY, // Ready: Báo hiệu slave đã sẵn sàng
    output reg [31:0] PRDATA, // Read Data: Bus dữ liệu đọc ra
    output wire    PSLVERR // Slave Error: Báo lỗi
);

    reg signed [7:0] reg_a;
    reg signed [7:0] reg_b;
    wire signed [15:0] mult_result;

    signed_multiplier_8bit u_core (
        .a(reg_a),
        .b(reg_b),
        .p(mult_result)
    );

    wire apb_write = PSEL & PENABLE & PWRITE;

    always @(posedge PCLK or negedge PRESETn) begin
        if (!PRESETn) begin
            reg_a <= 8'sd0;
            reg_b <= 8'sd0;
        end else if (apb_write) begin
            case (PADDR)
                8'h00: reg_a <= PWDATA[7:0];
                8'h04: reg_b <= PWDATA[7:0];
            endcase
        end
    end
end

```

```

end
always @(*) begin
    PRDATA = 32'd0;

    if (PSEL && !PWRITE) begin
        case (PADDR)
            8'h00: PRDATA = {24'd0, reg_a};
            8'h04: PRDATA = {24'd0, reg_b};
            8'h08: PRDATA = {{16{mult_result[15]}}, mult_result};
            default: PRDATA = 32'd0;
        endcase
    end
end

assign PREADY = 1'b1;
assign PSLVERR = 1'b0; // Không thiết kế bẫy lỗi ở đây

endmodule

```

Bảng 2 Module APB Slave

```

module apb_master (
    input wire    PCLK,
    input wire    PRESETn,

    input wire    start_write,
    input wire    start_read,
    input wire [7:0] addr_in,
    input wire [31:0] data_in,
    output reg [31:0] data_out,
    output reg     done,

    output reg [7:0] PADDR,
    output reg     PSEL,
    output reg     PENABLE,
    output reg     PWRITE,
    output reg [31:0] PWDATA,
    input wire [31:0] PRDATA,
    input wire     PREADY
);

localparam IDLE  = 2'b00;
localparam SETUP = 2'b01;
localparam ACCESS = 2'b10;

reg [1:0] state;

always @(posedge PCLK or negedge PRESETn) begin

```

```

if (!PRESETn) begin
    state <= IDLE;
    PSEL <= 0;
    PENABLE <= 0;
    done <= 0;
end else begin
    case (state)
        IDLE: begin
            done <= 0;
            if (start_write || start_read) begin
                state <= SETUP;
                PADDR <= addr_in;
                PSEL <= 1;
                PWRITE <= start_write;
                PWDATA <= data_in;
            end
        end
        SETUP: begin
            state <= ACCESS;
            PENABLE <= 1;
        end
        ACCESS: begin
            if (PREADY) begin
                if (!PWRITE) data_out <= PRDATA;
                PSEL <= 0;
                PENABLE <= 0;
                done <= 1;
                state <= IDLE;
            end
        end
    endcase
end
end
endmodule

```

Bảng 3 Module APB Master

```

module apb_system (
    input wire PCLK,
    input wire PRESETn,

    input wire    sw_wr,
    input wire    sw_rd,
    input wire [7:0] sw_addr,
    input wire [31:0] sw_data,
    output wire [31:0] sw_out,

```



```

.PCLK(PCLK), .PRESETn(PRESETn),
.sw_wr(sw_wr), .sw_rd(sw_rd),
.sw_addr(sw_addr), .sw_data(sw_data),
.sw_out(sw_out), .sw_done(sw_done)
);

always #5 PCLK = ~PCLK;

task apb_write(input [7:0] addr, input [31:0] data);
begin
    @(posedge PCLK);
    sw_addr = addr;
    sw_data = data;
    sw_wr = 1;
    wait(sw_done);
    @(posedge PCLK) sw_wr = 0;
end
endtask

task apb_read(input [7:0] addr, output [31:0] data_out);
begin
    @(posedge PCLK);
    sw_addr = addr;
    sw_rd = 1;
    wait(sw_done);
    data_out = sw_out;
    @(posedge PCLK) sw_rd = 0;
end
endtask

task test_multiply(input signed [7:0] a, input signed [7:0] b);
    reg signed [15:0] theory_val;

    time start_time;
    time end_time;

    begin
        start_time = $time;

        theory_val = a;
        theory_val = theory_val * b;

        apb_write(8'h00, {{24{a[7]}}, a});
        apb_write(8'h04, {{24{b[7]}}, b});

        apb_read(8'h08, read_val);
    end
endtask

```

```

        end_time = $time;

        $display("[Time: %6t ns -> %6t ns] (Mat %3t ns) | Test: %4d x %4d = %6d | Ly
thuyet = %6d",
                start_time, end_time, (end_time - start_time), a, b, $signed(read_val),
theory_val);
        end
    endtask

initial begin
    PCLK = 0; PRESETn = 0;
    sw_wr = 0; sw_rd = 0; sw_addr = 0; sw_data = 0;

    #15 PRESETn = 1;
    #10;

    $display("=====");
    $display("          BAT DAU CHAY TEST CORNER CASES          ");
    $display("=====");

    test_multiply(8'sd15, -8'sd4);
    test_multiply(8'sd127, 8'sd127);
    test_multiply(-8'sd128, -8'sd128);
    test_multiply(-8'sd128, 8'sd127);
    test_multiply(8'sd0, -8'sd55);
    test_multiply(8'sd1, -8'sd100);

    $display("=====");
    $display("          HOAN THANH          ");
    $display("=====");

    #20 $finish;
end
endmodule

```

Bảng 5 Module Testbench

===== BAT DAU CHAY TEST CORNER CASES =====

```

[Time: 25000 ns -> 135000 ns] (Mat 110000 ns) | Test: 15 x -4 = -60 | Ly thuyet = -60
[Time: 135000 ns -> 255000 ns] (Mat 120000 ns) | Test: 127 x 127 = 16129 | Ly thuyet = 16129
[Time: 255000 ns -> 375000 ns] (Mat 120000 ns) | Test: -128 x -128 = 16384 | Ly thuyet = 16384
[Time: 375000 ns -> 495000 ns] (Mat 120000 ns) | Test: -128 x 127 = -16256 | Ly thuyet = -16256
[Time: 495000 ns -> 615000 ns] (Mat 120000 ns) | Test: 0 x -55 = 0 | Ly thuyet = 0
[Time: 615000 ns -> 735000 ns] (Mat 120000 ns) | Test: 1 x -100 = -100 | Ly thuyet = -100
=====

```

- Giải thích thời gian xử lý:
 - Số lệnh cần xử lý: 3 lệnh, gồm: ghi A, ghi B, đọc kết quả
 - Thời gian cụ thể xử lý từng lệnh (tại cạnh lên xung clock):
 - Chu kì 1: Testbench bật `sw_wr = 1`
 - Chu kì 2: FSM trong APB Master nhảy qua trạng thái SETUP
 - Chu kì 3: FSM trong APB Master qua trạng thái ACCESS
 - Chu kì 4: APB Master nhận được cờ PREADY, bật cờ `done = 1` và chuyển về trạng thái IDLE. Testbench nhận cờ `done`, bật cờ `sw_wr = 0`.
 - Tổng cộng: 120ns cho cả 3 lệnh
 - Lệnh đầu chỉ tốn 110ns vì:


```
#15 PRESETn = 1;
#10;
```
- ⇒ 25ns (thời gian bắt đầu) trùng với cạnh lên xung clock → thực hiện luôn chu kì 1 của lệnh đầu tiên